

WEEK-10

Aim

To develop a Student Registration web application using the MERN stack (MongoDB, Express.js, React.js, Node.js) that lets users register student details and perform Create, Read, Update, and Delete (CRUD) operations, with the data stored in MongoDB through Mongoose.

Technologies Required

- Frontend: React.js
- Backend: Node.js and Express.js
- Database: MongoDB (database studentdb, collection students)
- Database library: Mongoose
- Communication: Fetch API

Description

This is a full-stack web application with three layers that work together:

1. **React frontend:** shows the registration form and the "Registered Students" table.
2. **Express.js backend:** a REST API server that receives requests from React.
3. **MongoDB database:** stores the student records, and Mongoose connects Express to it.

The registration form has these controls:

Field	Control
Name	Text box
Email	Email input
Password	Password input
Gender	Radio buttons
Country	Dropdown
Languages	Checkboxes
Register / Reset	Buttons

Concepts used:

- **useState():** holds the form data, the list of students, the status message, and the ID of the record being edited.
- **Controlled inputs:** text, email, password, radio, select, and checkbox values all come from state. The languages checkboxes are stored as an array.
- **useEffect():** fetches all students from the server when the page first loads.
- **Fetch API:** sends GET, POST, PUT, and DELETE requests from React to Express.
- **Express routes:** each route maps to one CRUD operation.
- **Mongoose schema and model:** define the structure of a student document (name, email, password, gender, country, languages) and provide methods such as find(), save(), findByIdAndUpdate(), and findByIdAndDelete().
- **CORS and express.json():** CORS lets the React app (a different port) call the server, and express.json() lets the server read JSON request bodies.
- **Conditional rendering:** shows the success or error message, and switches the button between "Register" and "Update" during editing.
- **List rendering:** .map() builds the table rows, each with Edit and Delete buttons.

REST API

Method	URL	Purpose
GET	/students	Retrieve all students
POST	/students	Register a new student
PUT	/students/:id	Update a student
DELETE	/students/:id	Delete a student

Program Code

App.jsx:

```
import { useEffect, useState } from "react";

const API = "http://localhost:5000";

function App() {

  const emptyForm = {

    name: "",

    email: "",

    password: "",

    gender: "",
```

```
country: "",
languages: []
};

const [form, setForm] = useState(emptyForm);
const [students, setStudents] = useState([]);
const [message, setMessage] = useState("");
const [editId, setEditId] = useState("");

// Get students when page loads
useEffect(() => {
  loadStudents();
}, []);

function loadStudents() {
  fetch(API + "/students")
    .then(response => response.json())
    .then(data => {
      setStudents(data);
    })
    .catch(error => {
      console.log(error);
      setMessage("Could not connect to server");
    });
}

// Handles text, email, password, radio and select
function handleChange(event) {
  const { name, value } = event.target;
  setForm({
    ...form,
    [name]: value
  });
}

// Handles language checkboxes
function handleLanguage(event) {
```

```

const value = event.target.value;

const checked = event.target.checked;

let languages = form.languages;

if (checked) {
  languages = [...languages, value];
} else {
  languages = languages.filter(item => item !== value);
}

setForm({
  ...form,
  languages: languages
});
}

// Register or update student
function saveStudent(event) {
  event.preventDefault();

  let url = API + "/students";

  let method = "POST";

  if (editId !== "") {
    url = API + "/students/" + editId;
    method = "PUT";
  }

  fetch(url, {
    method: method,
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(form)
  })
  .then(response => response.json())
  .then(data => {

```

```

    setMessage(data.message);
    setForm(emptyForm);
    setEditId("");
    loadStudents();
  })
  .catch(error => {
    console.log(error);
    setMessage("Could not connect to Express");
  });
}

// Put selected student data into form
function editStudent(student) {
  setForm({
    name: student.name,
    email: student.email,
    password: student.password,
    gender: student.gender,
    country: student.country,
    languages: student.languages
  });

  setEditId(student._id);
  setMessage("Edit the data and click Update");
}

// Delete student
function deleteStudent(id) {
  fetch(API + "/students/" + id, {
    method: "DELETE"
  })
  .then(response => response.json())
  .then(data => {
    setMessage(data.message);
  });
}

```

```
    loadStudents();

  });
}

// Clear form
function resetForm() {
  setForm(emptyForm);
  setEditId("");
  setMessage("");
}

return (
  <div>
    <h1>Student Registration</h1>
    <form onSubmit={saveStudent}>
      <label>Name</label>
      <br />
      <input
        name="name"
        value={form.name}
        onChange={handleChange}
        required
      />
      <br /><br />
      <label>Email</label>
      <br />
      <input
        type="email"
        name="email"
        value={form.email}
        onChange={handleChange}
        required
      />
      <br /><br />
    </form>
  </div>
);
```

```
<label>Password</label>
```

```
<br />
```

```
<input
```

```
  type="password"
```

```
  name="password"
```

```
  value={form.password}
```

```
  onChange={handleChange}
```

```
  required
```

```
<br /><br />
```

```
<label>Gender</label>
```

```
<br />
```

```
<input
```

```
  type="radio"
```

```
  name="gender"
```

```
  value="Male"
```

```
  checked={form.gender === "Male"}
```

```
  onChange={handleChange}
```

```
Male
```

```
<input
```

```
  type="radio"
```

```
  name="gender"
```

```
  value="Female"
```

```
  checked={form.gender === "Female"}
```

```
  onChange={handleChange}
```

```
Female
```

```
<br /><br />
```

```
<label>Country</label>
```

```
<br />
```

```
<select
```

```
name="country"
value={form.country}
onChange={handleChange}
required
>
<option value="">Select Country</option>
<option value="India">India</option>
<option value="USA">USA</option>
<option value="UK">UK</option>
<option value="Australia">Australia</option>
</select>
<br /><br />
<label>Languages</label>
<br />
<input
  type="checkbox"
  value="English"
  checked={form.languages.includes("English")}
  onChange={handleLanguage}
/>
English
<input
  type="checkbox"
  value="Hindi"
  checked={form.languages.includes("Hindi")}
  onChange={handleLanguage}
/>
Hindi
<input
  type="checkbox"
  value="Telugu"
  checked={form.languages.includes("Telugu")}
```

```

    onChange={handleLanguage}
  />
  Telugu
  <br /><br />
  <button type="submit">
    {editId === "" ? "Register" : "Update"}
  </button>
  <button type="button" onClick={resetForm}>
    Reset
  </button>
</form>
<p>{message}</p>
<hr />
<h2>Registered Students</h2>
<table border="1" cellPadding="8">
  <thead>
    <tr>
      <th>Name</th>
      <th>Email</th>
      <th>Gender</th>
      <th>Country</th>
      <th>Languages</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    {students.map(student => (
      <tr key={student._id}
        <td>{student.name}</td>
        <td>{student.email}</td>
        <td>{student.gender}</td>
        <td>{student.country}</td>

```

```

        <td>{student.languages.join(", ")}</td>

        <td>

        <button onClick={() => editStudent(student)}>

            Edit

        </button>

        <button onClick={() => deleteStudent(student._id)}>

            Delete

        </button>

        </td>

    </tr>

    )}

</tbody>

</table>

</div>

);

}

```

export default App;

server.js:

```

import express from "express";
import mongoose from "mongoose";
import cors from "cors";

const app = express();

app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose
    .connect("mongodb://127.0.0.1:27017/studentdb")
    .then(() => {
        console.log("MongoDB connected");
    })

```

```
    })  
    .catch((error) => {  
        console.log("MongoDB connection error:", error.message);  
    });  
  
// Schema  
const studentSchema = new mongoose.Schema({  
    name: String,  
    email: String,  
    password: String,  
    gender: String,  
    country: String,  
    languages: [String]  
});  
  
// Model  
const Student = mongoose.model("Student", studentSchema);  
  
// Test route  
app.get("/", (req, res) => {  
    res.send("Express server is working");  
});  
  
// READ  
app.get("/students", async (req, res) => {  
    const students = await Student.find();  
    res.json(students);  
});  
  
// CREATE  
app.post("/students", async (req, res) => {  
    const student = new Student(req.body);
```

```
await student.save();

res.json({
  message: "Registration successful"
});
});

// UPDATE
app.put("/students/:id", async (req, res) => {
  await Student.findByIdAndUpdate(
    req.params.id,
    req.body
  );

  res.json({
    message: "Student updated successfully"
  });
});

// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);

  res.json({
    message: "Student deleted successfully"
  });
});

// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

CRUD workflow

- **Create:** the user fills in the form and clicks Register. React sends a POST request to Express, Mongoose saves the document in MongoDB, and the message "Registration successful" appears.
- **Read:** React sends a GET request, Express retrieves the documents, and they appear in the table.
- **Update:** clicking Edit loads that record into the form. After the user changes it and clicks Update, a PUT request updates the document in MongoDB.
- **Delete:** clicking Delete sends a DELETE request. The record is removed from MongoDB and the table refreshes.

Data flow: React UI → fetch() → Express.js → Mongoose → MongoDB → Express.js → React UI

Result

The Student Registration application was developed using the MERN stack. It successfully performs Create, Read, Update, and Delete operations on student records stored in MongoDB, with every operation driven from the React UI.

Student Registration

Name

Email

Password

Gender
☐ Male ☐ Female

Country

Languages
☐ English ☐ Hindi ☐ Telugu

Registered Students

Name	Email	Gender	Country	Languages	Actions
susmitha	unnamsmitha.24.csm@anits.edu.in	Female	India	English, Hindi, Telugu	Edit Delete